

Tracking in Delphes

FastTrackCovariance Module

January 31st, 2025

Dimitris Ntounis

Stanford University &
SLAC National Accelerator Laboratory



- Tracking in [Delphes](#) has traditionally been done parametrically: obtain parametric descriptions of track momentum resolution as a function of p_T, η from full-sim study and apply description to smear the momentum for charged particles in Delphes
- New [TrackCovariance](#) module has been implemented in Delphes, which allows the track parameters (and the corresponding cov. Matrix) to be obtained on a per-particle level.

$$\vec{\alpha} = (D, \varphi_0, C, z_0, \lambda)$$

Defined in the point of closest approach (PCA) of the track to the z-axis; D is the signed transverse distance of the PCA from the z-axis, φ_0 is the track azimuthal angle, C is the signed half curvature, z_0 the \$z\$-coordinate of the PCA and λ the cotangent of the track polar angle.

$$\chi^2 = (\vec{d} - \vec{d}^*)^T S^{-1} (\vec{d} - \vec{d}^*)$$
$$\vec{d} \simeq \vec{d}_0 + \frac{\partial \vec{d}}{\partial \vec{\alpha}} \delta \vec{\alpha} = \vec{d}_0 + A \delta \vec{\alpha},$$

$$\chi^2 = (\vec{d}_0 - \vec{d}^* + A \delta \vec{\alpha})^T S^{-1} (\vec{d}_0 - \vec{d}^* + A \delta \vec{\alpha})$$

$$C^{-1} = \frac{1}{2} \frac{\partial^2 \chi^2}{\partial \vec{\alpha} \partial \vec{\alpha}} = A^T S^{-1} A$$

$$S_{ij} = \sigma_i^2 \delta_{ij} + M_{ij}$$

Contributions to covariance matrix from both intrinsic detector resolution & multiple scattering

<https://arxiv.org/pdf/2202.03285>

- Clone this repo: <https://github.com/dntounis/FastTrackCovariance>
- The idea is the following:
 - Specify tracking system configuration in txt file: layer type (barrel/disk), length, distance from IP, width, single-hit resolution
 - The program parses the txt file to create the tracking geometry (can visualize it and also print material budget)
 - The tracking geometry is then used along with an input particle gun to estimate the resolution on the various track parameters as a function of p_T, η
- Same module can be plugged into Delphes, so once txt file for tracking geometry is obtained, can just plug-and-play in a Delphes card.

FastTrackCovariance Tutorial



```
cd /fs/ddn/sdf/group/atlas/d/dntounis
mkdir FastTrackCovariance_tutorial && git clone https://github.com/
dntounis/FastTrackCovariance.git FastTrackCovariance_tutorial
```

```
cd /fs/ddn/sdf/group/atlas/d/dntounis
export HOME=/fs/ddn/sdf/group/atlas/d/dntounis
setupATLAS -c el9 -b
```

```
lsetup "root 6.32.02-x86_64-el9-gcc13-opt"
```

```
cd FastTrackCovariance_tutorial/geometry_scripts
root
```

```
.L SolGeomSiD.cxx
SolGeom()
SolGeom solGeomObj
solGeomObj.GeoPrint("GeoSiD_TEST.txt")
solGeomObj.Draw()
```

```
#This should create the detector geometry!
```

Print SiD material budget

```
# cd to FastTrackCovariance_tutorial
```

```
root
```

```
.L plot_materialSiD.c
plot_materialSiD()
```

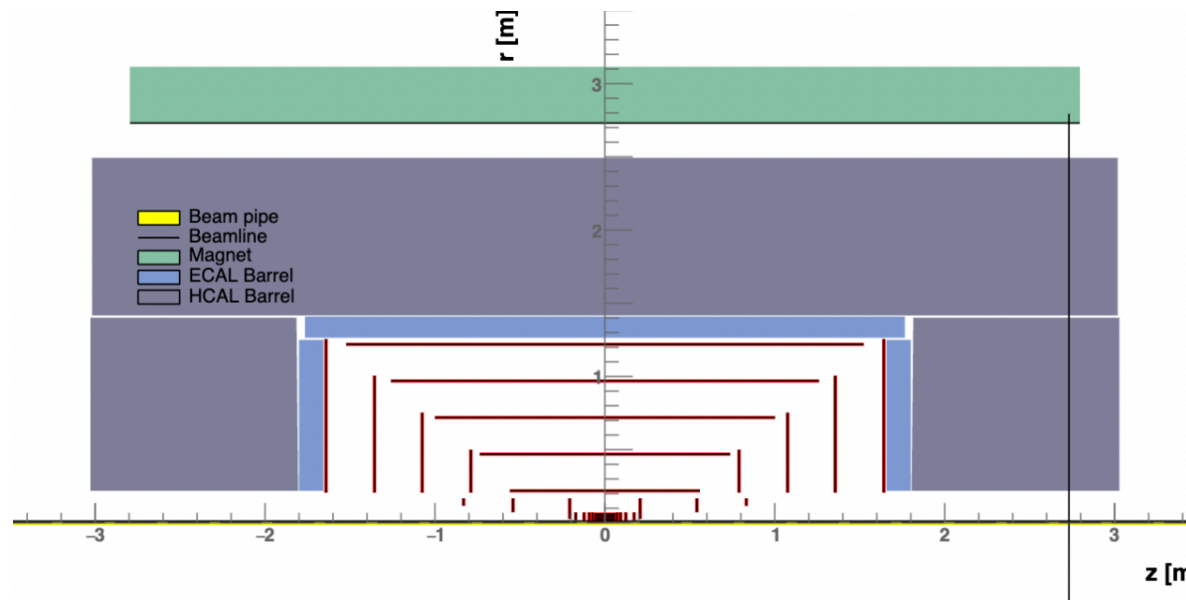
Compare tracking resolution for IDEA, CLD and SiD

```
# cd to FastTrackCovariance_tutorial
```

```
root
```

```
.L LoadAll.c
LoadAll("IDEA")
```

```
.L CompGeom.c
CompGeom(50) //50 deg is the track angle
```



If you want to implement your own geometry, open and edit SolGeomSiD.cxx (or cp it and change detector name :))

```
//
// Vertex detector (inner)
if (fEnable[1])
{
    const Int_t NlVtx = 5; // 5 vertex barrel pixel layers
    Double_t rVtx[NlVtx] = { 1.4, 2.2, 3.5, 4.8, 6.0 }; // Vertex layer radii in cm
    Double_t lVtx[NlVtx] = { 6.3, 6.3, 6.3, 6.3, 6.3 }; // Vertex layer half length in cm
    for (Int_t i = 0; i < NlVtx; i++)
    {
        ftyLay[fNlay] = 1; // Layer type 1 = R (barrel) or 2 = z (forward)
        fLyLabl[fNlay] = "VTX"; // Layer label
        fxMin[fNlay] = -lVtx[i] * 1.e-2; // Minimum dimension z for barrel or R for forward
        fxMax[fNlay] = lVtx[i] * 1.e-2; // Maximum dimension z for barrel or R for forward
        frPos[fNlay] = rVtx[i] * 1.e-2; // R/z location of layer
        fthLay[fNlay] = 50.E-6; // Thickness (meters) - Jim: assume 50 microns thickness (A
        frlLay[fNlay] = 9.370e-2; // Radiation length (meters) for Si: https://pdg.l
        fnmLay[fNlay] = 2; // Number of measurements in layers (1D or
        fstLayU[fNlay] = 0; // Stereo angle (rad) - 0(pi/2) = axial(z)
        fstLayL[fNlay] = TMath::Pi() / 2.; // Stereo angle (rad) - 0(pi/2) = axial(z) layer -
        fsgLayU[fNlay] = 3.E-6; // Resolution Upper side (meters) - 0 = no measurem
        fsgLayL[fNlay] = 3.E-6; // Resolution Lower side (meters) - 0 = no measurem
        fflLay[fNlay] = kTRUE; // measurement flag = T, scattering only = F
        fNlay++; fBlay++;
        fNm++;
    }
}
```